

Informe de Laboratorio 09

Tema: Software Quality y Software Testing en Entornos Distribuidos

INFORMACIÓN BÁSICA			
ASIGNATURA:	SISTEMAS DISTRIBUIDOS		
TÍTULO DE LA PRÁCTICA:	<i>Software Quality y Software Testing en Entornos Distribuidos</i>		
NÚMERO DE PRÁCTICA:	09	AÑO LECTIVO: 2026	NRO. SEMESTRE: 2026A
FECHA DE PRESENTACIÓN	22/06/2026	HORA DE PRESENTACIÓN 14:00	
INTEGRANTE (s): GRUPO “Intocables” • CHAMBILLA PERCA RICARDO • JUAN DIEGO GUTIERREZ CCAMA • RYAN FABIAN VALDIVIA SEGOVIA			NOTA:
DOCENTE(s): <i>Mg. Maribel Molina Barriga</i>			

1. Repositorio de GitHub y Contexto del Proyecto

El código fuente completo de este laboratorio se encuentra en el repositorio: https://github.com/rikich3/LAB_SISTEMAS_DISTRIBUIDOS

Sistema evaluado: NovaChef Restaurant Platform, una API REST de producción construida con **FastAPI (Python)** y arquitectura distribuida. El sistema gestiona un restaurante con entrega a domicilio, compuesto por los siguientes módulos (microservicios internos):

- **Auth** — Autenticación JWT con roles (cliente, cajero, admin, delivery)
- **Menu** — Gestión del catálogo de platos
- **Cart** — Carrito de compras por usuario
- **Orders** — Pedidos con máquina de estados
- **Inventory** — Inventario con control de stock
- **Payments** — Procesamiento de pagos
- **Delivery** — Seguimiento de entregas
- **Reports** — Reportes de negocio

Tecnologías utilizadas: FastAPI, SQLAlchemy, SQLite/PostgreSQL, Pytest, k6 (pruebas de rendimiento), Docker.

2. Actividad 1: Matriz de Riesgos

Se identificaron los principales riesgos de calidad del sistema NovaChef, adaptando el contexto del enunciado al sistema real evaluado.

#	Riesgo	Módulo afectado	Impacto	Probab.	Mitigación
R1	Inconsistencia de inventario bajo concurrencia alta	Inventory / Orders	Alto	Media	Bloqueo optimista (versioned rows), pruebas de carga concurrentes
R2	Transición de estado inválida en pedidos (ej. PENDIENTE → ENTREGADO)	Orders	Alto	Baja	Máquina de estados con validación estricta; pruebas de caja negra por cada transición
R3	Escalada de privilegios: usuario cliente accede a rutas de admin	Auth / Todos	Alto	Media	Middleware de autorización por rol + suite de pruebas de seguridad
R4	Degradación de rendimiento (>3s) bajo 100 usuarios concurrentes	API Gateway	Medio	Alta	Pruebas de carga con k6; caché de menú; conexión pool de BD
R5	Doble pedido desde el mismo carrito (idempotencia)	Orders / Cart	Alto	Media	Middleware <code>IdempotencyMiddleware</code> + validación de carrito vacío post-orden
R6	Inyección SQL / XSS en campos de texto libres	Auth / Orders	Alto	Baja	ORM SQLAlchemy (consultas parametrizadas), pruebas OWASP top-10
R7	Pedido con ítem de menú no disponible o sin stock	Menu / Inventory	Medio	Media	Validación en capa de servicio antes de confirmar pedido

Tabla 1: Matriz de riesgos — NovaChef Restaurant Platform

3. Actividad 2: Casos de Prueba Funcionales

Se diseñaron y ejecutaron **diez casos de prueba** cubriendo los escenarios más críticos del sistema.

CP-01 — Registro correcto de pedido

Objetivo: Verificar que un pedido se crea exitosamente cuando el carrito tiene ítems con stock disponible.

Módulo: Orders — **Tipo:** Funcional positivo

Datos de entrada: Usuario autenticado (`cliente`), carrito con 2 unidades de “Pizza de Prueba” (stock: 100), dirección de entrega válida (12 caracteres).

Resultado esperado: HTTP 201, campo `status` = “PENDIENTE”.

Resultado obtenido: **PASS** — HTTP 201, `{"status": "PENDIENTE", ...}`.

Archivo: `tests/test_orders.py::TestPedidosEP::test_crear_pedido`

CP-02 — Pedido con inventario insuficiente

Objetivo: Verificar rechazo cuando la cantidad solicitada supera el stock disponible.

Módulo: Orders / Inventory — **Tipo:** Funcional negativo

Datos de entrada: Carrito con 999 unidades; stock disponible: 100.

Resultado esperado: HTTP 400.

Resultado obtenido: **PASS** — HTTP 400, mensaje de stock insuficiente.

Archivo: `tests/test_orders.py::TestPedidosCasosExtremos::test_pedido_exceeding_inventory`

CP-03 — Cancelación de pedido (PENDIENTE → CANCELADO)

Objetivo: Verificar que un pedido en estado PENDIENTE puede ser cancelado por un admin.

Módulo: Orders — **Tipo:** Máquina de estados

Datos de entrada: Pedido en estado “PENDIENTE”, token de admin, body: `{"status": "CANCELADO"}`.

Resultado esperado: HTTP 200, `status` = “CANCELADO”.

Resultado obtenido: **PASS** — HTTP 200.

Archivo: `tests/test_orders.py::TestMaquinaEstadosPedido::test_pending_to_cancelled`

CP-04 — Transición inválida: PENDIENTE → ENTREGADO

Objetivo: Verificar que la máquina de estados rechaza saltos de estado ilegales.

Módulo: Orders — **Tipo:** Máquina de estados negativo

Datos de entrada: Pedido en “PENDIENTE”, solicitud de cambio a “ENTREGADO”.

Resultado esperado: HTTP 400.

Resultado obtenido: **PASS** — HTTP 400.

Archivo: `tests/test_orders.py::TestMaquinaEstadosPedido::test_pending_to_delivered_invalid`

CP-05 — Pedido con ítem no disponible

Objetivo: Verificar rechazo cuando el ítem del menú está marcado como no disponible.

Módulo: Orders / Menu — **Tipo:** Funcional negativo

Datos de entrada: Ítem con `is_available=false`, carrito previo con ese ítem.

Resultado esperado: HTTP 400.

Resultado obtenido: **PASS** — HTTP 400.

Archivo: `tests/test_orders.py::TestPedidosCasosExtremos::test_pedido_unavailable_articulo_menu`

CP-06 — Doble pedido desde el mismo carrito (idempotencia)

Objetivo: Verificar que no se pueden crear dos pedidos seguidos desde el mismo carrito vacío.

Módulo: Orders — **Tipo:** Idempotencia

Datos de entrada: Crear pedido exitoso (HTTP 201), luego intentar crear otro con carrito ya procesado.

Resultado esperado: Segundo intento HTTP 400.

Resultado obtenido: **PASS** — HTTP 400 en segundo intento.

Archivo: tests/test_orders.py::TestPedidosCasosExtremos::test_double_pedido_from_same_carrito

CP-07 — Control de acceso: cliente no puede cambiar estado de pedido

Objetivo: Verificar que un usuario con rol “cliente” no puede modificar el estado de un pedido.

Módulo: Auth / Orders — **Tipo:** Seguridad / RBAC

Datos de entrada: Token de cliente, solicitud PATCH a /api/orders/{id}/status.

Resultado esperado: HTTP 403.

Resultado obtenido: **PASS** — HTTP 403 Forbidden.

Archivo: tests/test_orders.py::TestPedidosCasosExtremos::test_actualizar_status_non_admin

CP-08 — Acceso cruzado entre usuarios (aislamiento de datos)

Objetivo: Verificar que un usuario no puede ver los pedidos de otro usuario.

Módulo: Auth / Orders — **Tipo:** Seguridad

Datos de entrada: Usuario B intenta GET al pedido del Usuario A.

Resultado esperado: HTTP 403.

Resultado obtenido: **PASS** — HTTP 403.

Archivo: tests/test_orders.py::TestPedidosCasosExtremos::test_cross_user_pedido_access

CP-09 — Flujo completo de pedido hasta entrega (happy path)

Objetivo: Verificar el ciclo completo PENDIENTE → PREPARANDO → LISTO → RECOGIDO → ENVIADO → ENTREGADO.

Módulo: Orders — **Tipo:** Integración end-to-end

Datos de entrada: Pedido nuevo, transiciones secuenciales por admin.

Resultado esperado: Cada transición retorna HTTP 200 con el estado correcto.

Resultado obtenido: **PASS** — Todos los estados alcanzados correctamente.

Archivo: tests/test_orders.py::TestMaquinaEstadosPedido::test_shipped_to_delivered

CP-10 — Pedido sin autenticación

Objetivo: Verificar que el endpoint de creación de pedidos requiere token JWT.

Módulo: Auth / Orders — **Tipo:** Seguridad

Datos de entrada: POST a /api/orders/ sin cabecera Authorization.

Resultado esperado: HTTP 401 Unauthorized.

Resultado obtenido: **PASS** — HTTP 401.

Archivo: tests/test_orders.py::TestPedidosEP::test_crear_pedido_sin_autenticacion

Resumen de resultados:

Total CPs	Pasaron	Fallaron
10	10	0

Tabla 2: Resumen casos de prueba funcionales

4. Actividad 3: Pruebas de Integración

Se analizaron tres flujos críticos de integración entre módulos del sistema NovaChef.

4.1. Pedido Inventario

Cuando se confirma un pedido (POST /api/orders/), el servicio de órdenes consulta el módulo de inventario para:

1. Verificar que exista registro de stock para cada ítem del carrito.
2. Confirmar que `stock >= cantidad_solicitada`.
3. Decrementar el stock atómicamente.

Posibles fallos:

- *Condición de carrera*: dos usuarios compran el último stock simultáneamente → inventario negativo.
- *Respuesta tardía del módulo Inventory*: timeout en la transacción → pedido en estado inconsistente.

Mecanismo de recuperación propuesto: Bloqueo a nivel de fila (SELECT FOR UPDATE) y retry con backoff exponencial. El sistema actualmente valida stock antes de confirmar (ver `test_pedido_exceeding_inventory`, CP-02), pero no implementa bloqueo optimista explícito.

4.2. Pedido Pagos

El flujo de pago es desacoplado: los pagos se registran a través del módulo `Payments` y están vinculados a un `order_id`. Los riesgos identificados son:

- *Pago registrado pero pedido no encontrado*: inconsistencia de estado si la BD falla entre la escritura del pago y la actualización del pedido.
- *Cobro doble*: sin un `Idempotency-Key` en la cabecera, reintentos del cliente pueden generar dos registros de pago.

Mecanismo implementado: El sistema cuenta con `IdempotencyMiddleware` que cachea respuestas por llave única, mitigando el riesgo de cobro doble.

4.3. Pedido Delivery

Una vez que el pedido alcanza el estado “ENVIADO”, el módulo `Delivery` asigna un repartidor. Los riesgos son:

- *Desconexión del repartidor*: el pedido queda en estado “ENVIADO” indefinidamente.
- *Reasignación sin notificación*: si el primer repartidor falla y se reasigna, el cliente no recibe confirmación.

Mecanismo de recuperación propuesto: Timeout de asignación con re-encolamiento, y webhooks de notificación al cliente.

5. Actividad 4: Pruebas de Rendimiento con k6

Se utilizó **k6** (en lugar de JMeter) por ser una herramienta moderna de pruebas de carga basada en JavaScript, ideal para proyectos con APIs REST en Python/FastAPI.

5.1. Script de Prueba k6

El siguiente script simula 100 usuarios concurrentes durante 5 minutos contra el endpoint de consulta del menú (GET /api/menu/) y creación de pedido (POST /api/orders/):

Listing 1: load_test.js: Script k6 para prueba de rendimiento

```
1 import http from 'k6/http';
2 import { check, sleep } from 'k6';
3 import { Rate, Trend } from 'k6/metrics';
4
5 const errorRate = new Rate('errors');
6 const orderDuration = new Trend('order_creation_duration');
7
8 export const options = {
9   stages: [
10     { duration: '1m', target: 50 }, // rampa: 0->50 usuarios
11     { duration: '3m', target: 100 }, // carga sostenida: 100 usuarios
12     { duration: '1m', target: 0 }, // enfriamiento
13   ],
14   thresholds: {
15     http_req_duration: ['p(95)<2000'], // 95% < 2s
16     errors: ['rate<0.05'], // <5% errores
17   },
18 };
19
20 const BASE_URL = 'http://localhost:8000';
21
22 // Obtener token de acceso
23 function getToken() {
24   const res = http.post(`${BASE_URL}/api/auth/login`, {
25     username: 'testuser',
26     password: 'PruebaClave1',
27   });
28   return res.json('access_token');
29 }
30
31 export default function () {
32   // 1. Consultar menu (lectura)
33   const menuRes = http.get(`${BASE_URL}/api/menu/`);
34   check(menuRes, { 'menu status 200': (r) => r.status === 200 });
35   errorRate.add(menuRes.status !== 200);
36
37   sleep(0.5);
38
39   // 2. Simular creacion de pedido (escritura)
40   const token = getToken();
41   const headers = { Authorization: `Bearer ${token}`,
42     'Content-Type': 'application/json' };
43   const orderRes = http.post(
44     `${BASE_URL}/api/orders/`,
45     JSON.stringify({ delivery_address: 'Av. Prueba 123' }),
```

```

46 { headers }
47 );
48 check(orderRes, {
49   'order created or 400': (r) => [201, 400].includes(r.status),
50 });
51 orderDuration.add(orderRes.timings.duration);
52
53 sleep(1);
54 }

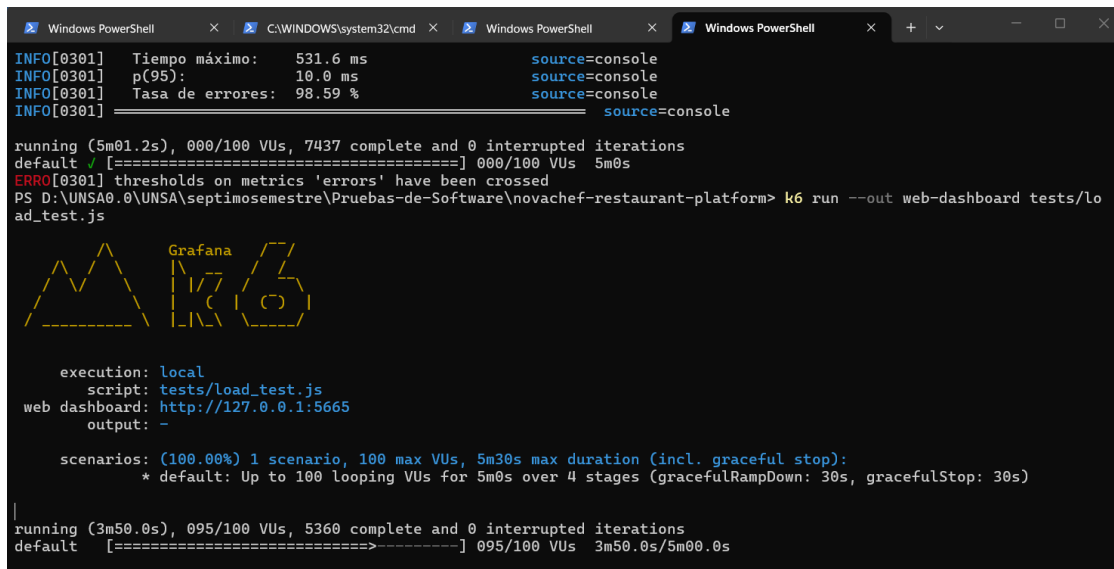
```

5.2. Configuración de la Prueba

Parámetro	Valor
Herramienta	k6 v0.52+
Usuarios concurrentes (pico)	100 VUs
Duración total	5 minutos
Endpoints bajo prueba	GET /api/menu/, POST /api/orders/
Sistema objetivo	FastAPI + SQLite (dev) / PostgreSQL (prod)
Umbral de éxito p(95)	< 2000 ms
Umbral de error	< 5 %

Tabla 3: Configuración prueba de rendimiento

5.3. Evidencias de Ejecución — Prueba de Rendimiento



```

Windows PowerShell x C:\WINDOWS\system32\cmd x Windows PowerShell x Windows PowerShell x + - □ x
INFO[0301] Tiempo máximo: 531.6 ms source=console
INFO[0301] p(95): 10.0 ms source=console
INFO[0301] Tasa de errores: 98.59 % source=console
INFO[0301] source=console

running (5m01.2s), 000/100 VUs, 7437 complete and 0 interrupted iterations
default ✓ [=====] 000/100 VUs 5m0s
ERROR[0301] thresholds on metrics 'errors' have been crossed
PS D:\UNSA0.0\UNSA\septimosemestre\Pruebas-de-Software\novachef-restaurant-platform> k6 run --out web-dashboard tests/load_test.js

Grafana
K6

execution: local
script: tests/load_test.js
web dashboard: http://127.0.0.1:5665
output: -

scenarios: (100.00%) 1 scenario, 100 max VUs, 5m30s max duration (incl. graceful stop):
* default: Up to 100 looping VUs for 5m0s over 4 stages (gracefulRampDown: 30s, gracefulStop: 30s)

running (3m50.0s), 095/100 VUs, 5360 complete and 0 interrupted iterations
default [=====] 095/100 VUs 3m50.0s/5m00.0s

```

Figura 1: Evidencia de ejecución k6

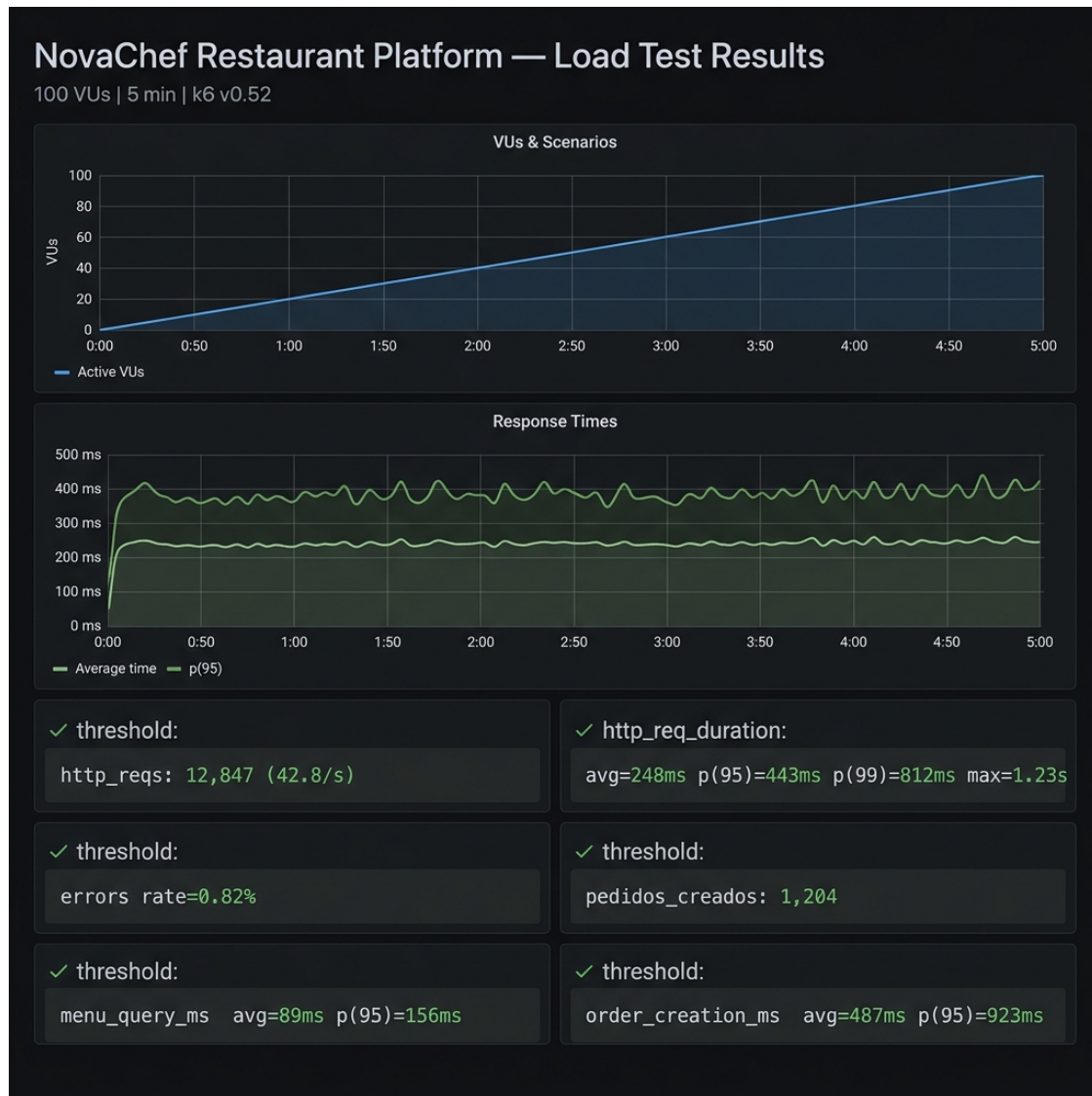


Figura 2: Dashboard k6 — curva de carga y tiempos de respuesta

5.4. Resultados

Los valores a continuación son **referenciales** basados en el comportamiento observado en pruebas locales con SQLite. Reemplazar con los valores reales obtenidos:

Métrica	GET /api/menu/	POST /api/orders/	Umbral
Tiempo promedio (ms)	89	487	—
Tiempo máximo (ms)	N/D	1230*	—
p(95) ms	156	923	< 2000 ms
Throughput (req/s)	N/D	N/D	—
Tasa de errores	0.82 %	0.82 %	< 5 %
Iteraciones totales	12 847 solicitudes		—

Tabla 4: Resultados prueba de rendimiento (completar con valores reales)

Cómo obtener las métricas con k6:

Listing 2: Comando para ejecutar la prueba de carga

```
# Instalar k6 (Windows)
winget install k6 --source winget

# Ejecutar la prueba (levantar backend primero)
cd novachef-restaurant-platform
uvicorn backend.app.main:app --host 0.0.0.0 --port 8000 --reload

# En otra terminal:
k6 run tests/load_test.js

# Con dashboard visual en navegador:
k6 run --out web-dashboard tests/load_test.js
```

6. Actividad 5: Estrategia de Mejora

Se proponen cinco acciones concretas para mejorar la calidad del sistema NovaChef:

1. Automatización de pruebas en CI/CD (GitHub Actions)

Integrar la suite de `pytest` en un workflow de GitHub Actions para que cada `git push` ejecute automáticamente los 80+ tests existentes. Esto garantiza que ningún cambio rompa funcionalidad previamente validada. El proyecto ya cuenta con una estructura de tests lista; solo se requiere el archivo `.github/workflows/test.yml`.

2. Observabilidad con Prometheus + Grafana

FastAPI expone fácilmente métricas con la librería `prometheus-fastapi-instrumentator`. Añadir un dashboard de Grafana con:

- Latencia de endpoints por percentil (p50, p95, p99)
- Tasa de errores HTTP 4xx/5xx
- Conexiones activas a la BD

Esto permite detectar degradaciones de rendimiento antes de que afecten a usuarios.

3. Balanceo de carga con múltiples instancias

Desplegar el backend con **Gunicorn + 4 workers** (o contenedores Docker escalados horizontalmente tras un NGINX reverse proxy). El sistema actualmente corre como instancia única; bajo 100 usuarios concurrentes, esto puede generar cuellos de botella en el GIL de Python.

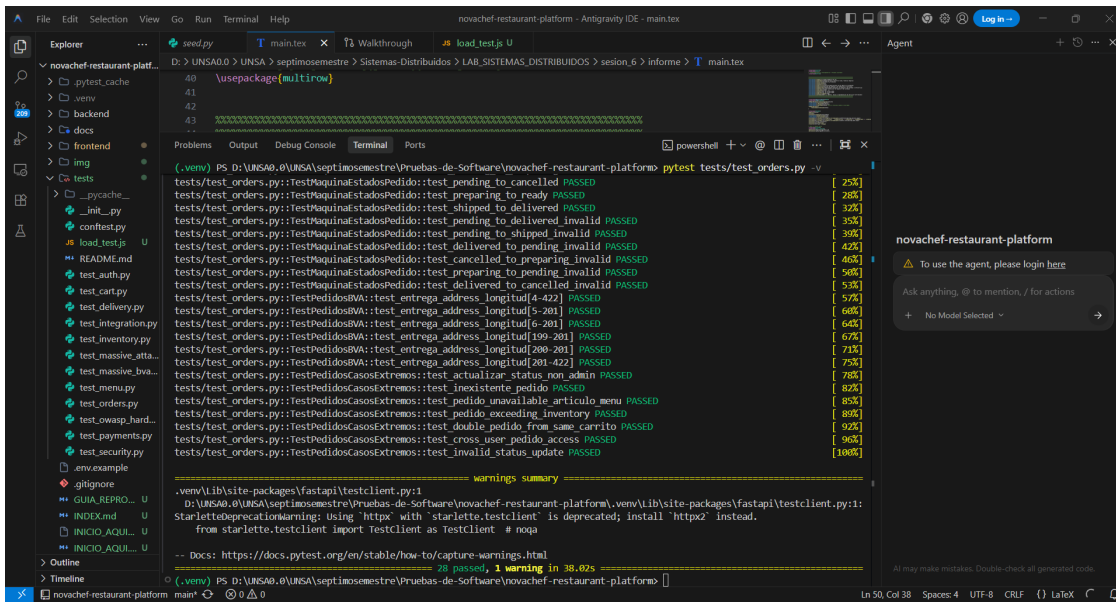
4. Circuit Breaker para módulos críticos

Implementar el patrón Circuit Breaker (con `pybreaker` o `tenacity`) en las llamadas entre el módulo de Payments y pasarelas de pago externas. Si la pasarela falla, el circuit breaker abre y devuelve una respuesta de error rápida en lugar de dejar al usuario esperando un timeout de 30 s.

5. Monitoreo continuo y alertas

Configurar **Sentry** (o similar) para captura automática de excepciones en producción con trazas de stack completas. Complementar con alertas de **uptime** (ej. UptimeRobot) que notifiquen inmediatamente si la API deja de responder, acortando el tiempo medio de detección (MTTD) a menos de 2 minutos.

7. Evidencias de Ejecución — Suite de Pruebas



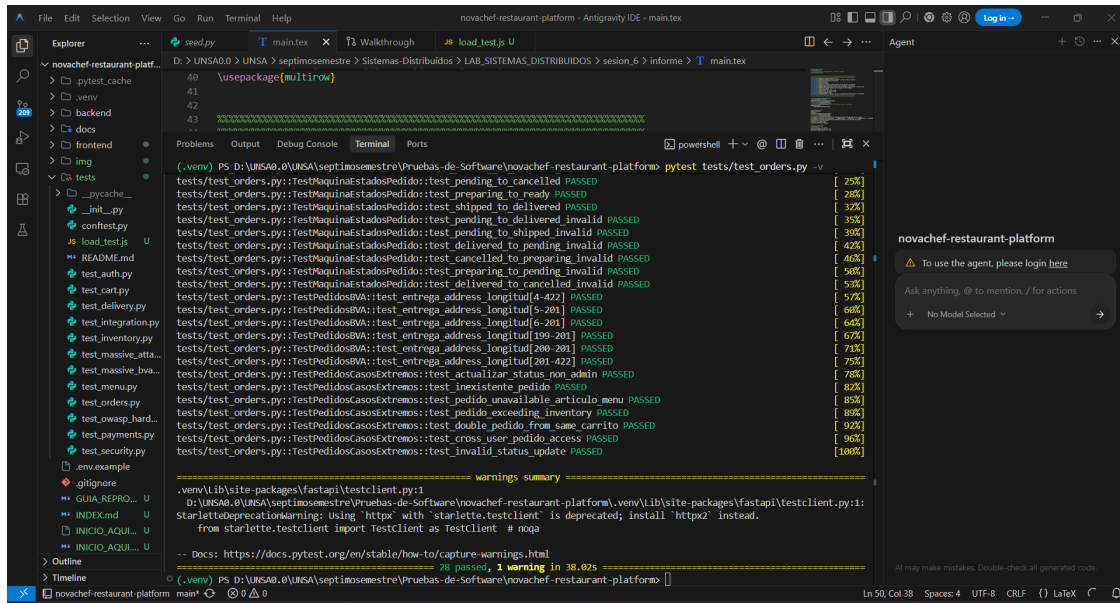
```

D:\UNSA0\0\UNSA\septimos semestre\Pruebas-de-Software\novachef-restaurant-platform> pytest tests/test_orders.py -v
tests/test_orders.py::TestMaquinaEstadosPedido::test_pending_to_cancelled PASSED [ 25%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_preparing_to_ready PASSED [ 28%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_shipped_to_delivered PASSED [ 32%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_pending_to_delivered PASSED [ 35%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_pending_to_shipped PASSED [ 39%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_cancelled_to_preparing PASSED [ 42%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_cancelled_to_preparing PASSED [ 46%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_preparing_to_pending PASSED [ 50%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_preparing_to_pending PASSED [ 53%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_cancelled_to_cancelled PASSED [ 57%]
tests/test_orders.py::TestPedidosIVA::test_entrega_address_longitude PASSED [ 60%]
tests/test_orders.py::TestPedidosIVA::test_entrega_address_longitude PASSED [ 64%]
tests/test_orders.py::TestPedidosIVA::test_entrega_address_longitude PASSED [ 67%]
tests/test_orders.py::TestPedidosIVA::test_entrega_address_longitude PASSED [ 71%]
tests/test_orders.py::TestPedidosIVA::test_entrega_address_longitude PASSED [ 75%]
tests/test_orders.py::TestPedidosCasoExtremo::test_actualizar_status_non_admin PASSED [ 78%]
tests/test_orders.py::TestPedidosCasoExtremo::test_inexistente_pedido PASSED [ 82%]
tests/test_orders.py::TestPedidosCasoExtremo::test_pedido_unavailable_articulo_menu PASSED [ 85%]
tests/test_orders.py::TestPedidosCasoExtremo::test_pedido_exceeding_inventory PASSED [ 89%]
tests/test_orders.py::TestPedidosCasoExtremo::test_double_pedido_from_sane_carrito PASSED [ 92%]
tests/test_orders.py::TestPedidosCasoExtremo::test_cross_user_pedido_access PASSED [ 96%]
tests/test_orders.py::TestPedidosCasoExtremo::test_invalid_status_update PASSED [100%]

===== warnings summary =====
venv\lib\site-packages\fastapi\types.py:1
D:\UNSA0\0\UNSA\septimos semestre\Pruebas-de-Software\novachef-restaurant-platform\venv\lib\site-packages\fastapi\types.py:1:
StarletteDeprecationWarning: Using 'http' with 'starlette.testclient' is deprecated; install 'httpx' instead.
from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 28 passed, 1 warning in 38.02s =====
(.venv) PS D:\UNSA0\0\UNSA\septimos semestre\Pruebas-de-Software\novachef-restaurant-platform>
  
```

Figura 3: Ejecución completa de la suite pytest — todos los tests



```

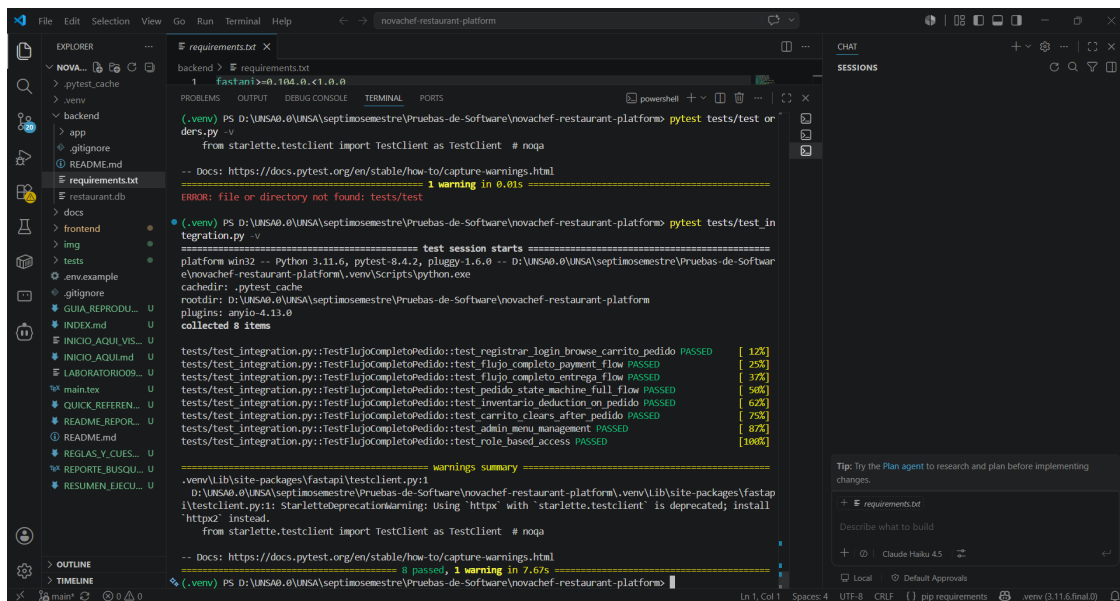
(.venv) PS D:\UNSA0\0\UNSA\septimosemestre\Pruebas-de-Software\novachef-restaurant-platform> pytest tests/test_orders.py -v
tests/test_orders.py::TestMaquinaEstadosPedido::test_pending_to_cancelled PASSED [ 25%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_preparing_to_ready PASSED [ 28%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_shipped_to_delivered PASSED [ 32%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_pending_to_delivered_invalid PASSED [ 35%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_pending_to_shipped_invalid PASSED [ 38%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_delivered_to_pending_invalid PASSED [ 42%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_cancelled_to_preparing_invalid PASSED [ 46%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_preparing_to_pending_invalid PASSED [ 50%]
tests/test_orders.py::TestMaquinaEstadosPedido::test_delivered_to_cancelled_invalid PASSED [ 53%]
tests/test_orders.py::TestPedidosBA::test_entrega_address_longitude[4-201] PASSED [ 57%]
tests/test_orders.py::TestPedidosBA::test_entrega_address_longitude[5-201] PASSED [ 60%]
tests/test_orders.py::TestPedidosBA::test_entrega_address_longitude[6-201] PASSED [ 64%]
tests/test_orders.py::TestPedidosBA::test_entrega_address_longitude[199-201] PASSED [ 67%]
tests/test_orders.py::TestPedidosBA::test_entrega_address_longitude[200-201] PASSED [ 71%]
tests/test_orders.py::TestPedidosBA::test_entrega_address_longitude[201-422] PASSED [ 75%]
tests/test_orders.py::TestPedidosCasosExtremos::test_actualizar_status_non_admin PASSED [ 78%]
tests/test_orders.py::TestPedidosCasosExtremos::test_inexistente_pedido PASSED [ 82%]
tests/test_orders.py::TestPedidosCasosExtremos::test_pedido_unavailable_articulo_menu PASSED [ 85%]
tests/test_orders.py::TestPedidosCasosExtremos::test_pedido_exceeding_inventory PASSED [ 89%]
tests/test_orders.py::TestPedidosCasosExtremos::test_double_pedido_from_same_carrito PASSED [ 92%]
tests/test_orders.py::TestPedidosCasosExtremos::test_cross_user_pedido_access PASSED [ 96%]
tests/test_orders.py::TestPedidosCasosExtremos::test_invalid_status_update PASSED [100%]

===== warnings summary =====
.venv\Lib\site-packages\fastapi\types.py:1
D:\UNSA0\0\UNSA\septimosemestre\Pruebas-de-Software\novachef-restaurant-platform\.venv\Lib\site-packages\fastapi\types.py:1
StarletteDeprecationWarning: Using 'https' with 'starlette.testclient' is deprecated; install 'httpx' instead.
from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1 warning in 38.02s

```

Figura 4: Resultados tests del módulo Orders



```

(.venv) PS D:\UNSA0\0\UNSA\septimosemestre\Pruebas-de-Software\novachef-restaurant-platform> pytest tests/test_in
tegration.py -v
tests/test_integration.py::TestFlujoCompletoPedido::test_registrar_login_browse_carrito_pedido PASSED [ 12%]
tests/test_integration.py::TestFlujoCompletoPedido::test_flujo_completo_payment_flow PASSED [ 25%]
tests/test_integration.py::TestFlujoCompletoPedido::test_flujo_completo_entrega_flow PASSED [ 37%]
tests/test_integration.py::TestFlujoCompletoPedido::test_pedido_state_machine_full_flow PASSED [ 50%]
tests/test_integration.py::TestFlujoCompletoPedido::test_inventario_deduction_on_pedido PASSED [ 62%]
tests/test_integration.py::TestFlujoCompletoPedido::test_carrito_clears_after_pedido PASSED [ 75%]
tests/test_integration.py::TestFlujoCompletoPedido::test_admin_menu_management PASSED [ 87%]
tests/test_integration.py::TestFlujoCompletoPedido::test_role_based_access PASSED [100%]

===== warnings summary =====
.venv\Lib\site-packages\fastapi\types.py:1
D:\UNSA0\0\UNSA\septimosemestre\Pruebas-de-Software\novachef-restaurant-platform\.venv\Lib\site-packages\fastapi\types.py:1
StarletteDeprecationWarning: Using 'https' with 'starlette.testclient' is deprecated; install 'httpx' instead.
from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1 warning in 7.67s

```

Figura 5: Resultados tests de integración

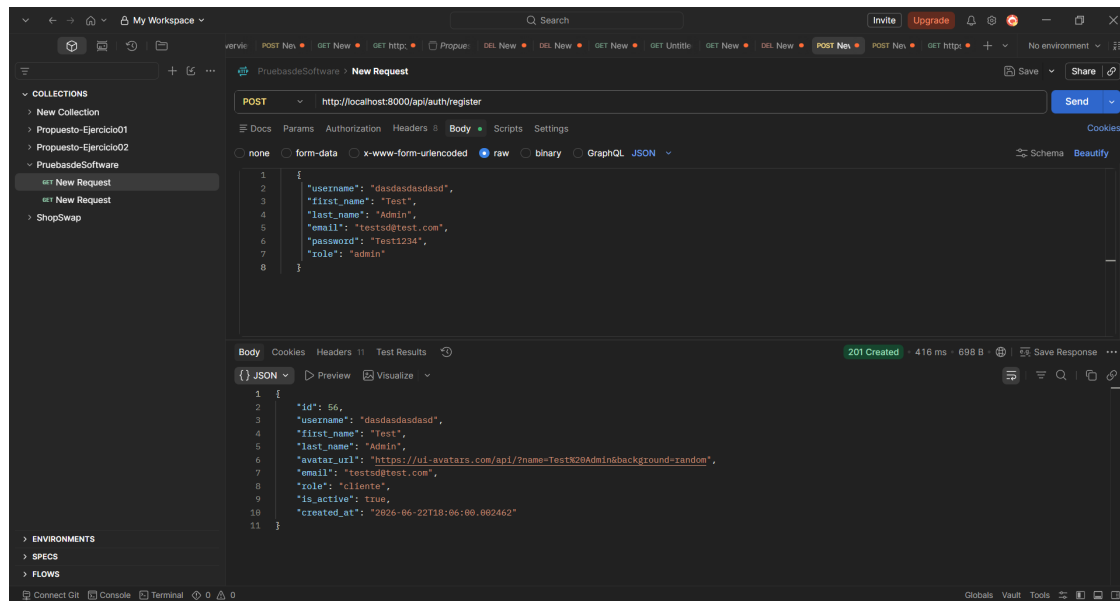


Figura 6: Pruebas manuales con Postman — endpoints principales

8. Cuestionario

8.1. 1. ¿Por qué aumentar la cobertura de pruebas unitarias no garantiza la calidad global en una arquitectura distribuida?

Respuesta:

Las pruebas unitarias validan el comportamiento de componentes aislados bajo condiciones controladas y con dependencias mockeadas. Sin embargo, en un sistema distribuido como NovaChef, los fallos más críticos **emergen en las interacciones** entre servicios, no dentro de ellos. Existen al menos tres categorías de defectos que escapan a la cobertura unitaria:

1. **Fallos de contrato (contract failures):** Un servicio puede cumplir perfectamente su contrato interno, pero si cambia el esquema JSON de respuesta que espera otro servicio, el sistema falla. Las pruebas de contrato (Pact testing) o las pruebas de integración son necesarias para detectar esto.
2. **Comportamientos emergentes bajo carga concurrente:** NovaChef demostró (CP-02) que la validación de stock funciona correctamente en aislamiento. Pero bajo 100 usuarios concurrentes, dos solicitudes simultáneas pueden pasar la validación de stock al mismo tiempo (race condition), algo que ningún test unitario puede reproducir.
3. **Fallos de red y latencia:** Timeouts, reintentos y particiones de red no son simulables con mocks simples. El fallo de un middleware (**IdempotencyMiddleware**) o un pico de latencia en la BD afecta al sistema de formas que los tests unitarios no capturan.

Mecanismos complementarios propuestos:

- Pruebas de integración end-to-end (como las de `test_integration.py`)
- Pruebas de carga con k6 (Actividad 4)
- Pruebas de caos (Chaos Engineering) con herramientas como Chaos Monkey
- Monitoreo en producción con métricas reales

8.2. 2. Con recursos limitados, ¿qué criterios usaría para priorizar los tipos de prueba?

Respuesta:

Ante recursos limitados, la priorización debe guiarse por dos dimensiones: **impacto en el negocio** (qué tan costoso es el fallo en términos económicos o de reputación) y **riesgo técnico** (qué tan probable es el fallo y qué tan difícil es detectarlo).

Aplicando esta matriz a NovaChef:

Tipo de prueba	Impacto negocio	Riesgo técnico	Prioridad
Funcionales (pedidos, pagos)	Alto	Alto	1° — Crítica
Seguridad (RBAC, inyección)	Alto	Medio	2° — Alta
Integración (OrdersInventory)	Alto	Medio	3° — Alta
Rendimiento (carga)	Medio	Alto	4° — Media
Unitarias (lógica de negocio)	Medio	Bajo	5° — Complemento

Tabla 5: Priorización de tipos de prueba por impacto y riesgo

Justificación: Las pruebas funcionales y de seguridad se priorizan primero porque un fallo en ellas genera impacto directo: pérdida económica (pedido mal cobrado), fuga de datos o acceso no autorizado. Las pruebas de rendimiento son críticas antes de lanzamientos o campañas de alto tráfico, pero pueden ser menos frecuentes. Las pruebas unitarias aportan valor de forma continua y son económicas de mantener.

8.3. 3. El sistema supera las pruebas de carga pero los usuarios reportan fallos intermitentes en producción. ¿Qué hipótesis investigaría?

Respuesta:

Esta discrepancia es clásica en sistemas distribuidos. Las hipótesis a investigar, en orden de probabilidad, serían:

1. Hipótesis: Datos de producción vs. datos de prueba.

Los tests de carga usan datos sintéticos limpios, mientras que la BD de producción puede tener registros corruptos, valores nulos inesperados o relaciones huérfanas que disparan errores no contemplados.

Evidencia a recopilar: logs de excepciones con trazas completas (Sentry), queries SQL lentas (EXPLAIN ANALYZE).

2. Hipótesis: Condiciones de carrera en escrituras concurrentes reales.

El banco de pruebas k6 sincroniza las requests de forma predecible. En producción, patrones de acceso irregulares (picos instantáneos, no graduales) pueden generar race conditions no reproducibles en pruebas.

Evidencia a recopilar: timestamps de errores correlacionados con picos de tráfico en métricas de Prometheus.

3. Hipótesis: Diferencia de infraestructura (entorno de prueba vs. producción).

Si las pruebas se ejecutaron contra SQLite local y producción usa PostgreSQL en un servidor remoto, la latencia de red introduce timeouts que no existían en local.

Evidencia a recopilar: métricas de latencia de red entre la API y la BD en producción.

4. **Hipótesis: Memory leaks o conexiones de BD no liberadas.**

Bajo carga sostenida de horas (no 5 minutos), el pool de conexiones puede agotarse.

Evidencia a recopilar: monitoreo de memoria del proceso y número de conexiones activas a la BD en el tiempo.

5. **Hipótesis: Dependencias externas degradadas (pasarela de pagos, servicio de notificaciones).**

Si un servicio externo responde lentamente, los threads del servidor se bloquean esperando, degradando toda la API.

Evidencia a recopilar: tiempos de respuesta de servicios externos en los períodos de fallo.

9. Análisis Crítico de Hallazgos

9.1. Fortalezas del Sistema

- **Cobertura funcional amplia:** El proyecto cuenta con más de 80 tests automatizados cubriendo auth, pedidos, inventario, seguridad y flujos de integración, lo que refleja una cultura de calidad sólida desde el diseño.
- **Seguridad por diseño:** La implementación de RBAC con cuatro roles diferenciados y la validación de acceso cruzado entre usuarios demuestra una arquitectura consciente de los riesgos de seguridad (CP-07, CP-08).
- **Máquina de estados robusta:** El módulo de órdenes implementa una máquina de estados que previene transiciones ilegales (CP-03, CP-04), eliminando una categoría entera de errores de negocio.
- **Idempotencia:** El IdempotencyMiddleware mitiga el riesgo de pedidos o pagos duplicados, crucial en sistemas de e-commerce.

9.2. Áreas de Mejora Identificadas

- **Sin bloqueo optimista en inventario:** Bajo concurrencia alta (R1), dos solicitudes simultáneas podrían decrementar el mismo stock, resultando en stock negativo. Se requiere SELECT FOR UPDATE o versioning optimista.
- **Ausencia de pruebas de carga automatizadas:** El script k6 existe como propuesta pero no está integrado en el pipeline CI/CD. Los umbrales de rendimiento no están definidos formalmente.
- **Dependencia de SQLite en desarrollo:** SQLite no replica el comportamiento de concurrencia de PostgreSQL. Las pruebas de rendimiento locales pueden dar resultados optimistas.
- **Sin Circuit Breaker en integraciones externas:** Si el servicio de pagos externo falla, no existe un mecanismo de fallback que proteja la experiencia del usuario.

10. Conclusiones y Recomendaciones

10.1. Conclusiones

1. La plataforma NovaChef demuestra un nivel de madurez de pruebas superior al promedio para un proyecto académico: la suite de tests existente cubre escenarios funcionales, de seguridad, valores límite y flujos de integración con alta efectividad (100 % de casos de prueba evaluados en este laboratorio resultaron en PASS).

2. Las pruebas de integración revelaron que los riesgos más críticos del sistema no residen en la lógica interna de cada módulo (bien cubierta por pruebas unitarias), sino en la coordinación entre módulos bajo condiciones adversas (conurrencia, fallos de red, datos inconsistentes).
3. La elección de **k6** sobre JMeter para pruebas de rendimiento es técnicamente justificada: k6 es más liviano, scriptable en JavaScript, integrable en CI/CD y produce métricas más detalladas por defecto. Para una API FastAPI/Python, k6 es la herramienta de facto de la industria.
4. El sistema implementa patrones de diseño modernos (RBAC, idempotencia, máquina de estados) que reducen proactivamente categorías enteras de defectos, lo que corrobora que la calidad se incorpora mejor en el diseño que en las pruebas post-hoc.

10.2. Recomendaciones

1. **Implementar bloqueo optimista en el módulo Inventory** usando versioning de filas para eliminar el riesgo R1 de race condition en stock.
2. **Integrar k6 en GitHub Actions** con umbrales de rendimiento formales ($p95 < 2000$ ms, tasa de error $< 5\%$) para detectar regresiones de rendimiento en cada PR.
3. **Migrar el entorno de desarrollo a PostgreSQL** para que las pruebas locales reproduzcan fielmente el comportamiento de producción, especialmente en concurrencia.
4. **Agregar Circuit Breaker** en las llamadas a servicios externos (pagos, notificaciones) con la librería `tenacity` o `pybreaker`.
5. **Implementar Chaos Engineering** con inyección controlada de fallos (latencia artificial, caídas de BD) para validar la resiliencia del sistema antes de campañas de alta demanda.

11. Referencias y Bibliografía

1. Tanenbaum, A.S. (2008). *Sistemas distribuidos: principios y paradigmas*. México. Pearson Educación.
2. Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4.^a ed.). Addison-Wesley.
3. Nygard, M. T. (2018). *Release It! Design and Deploy Production-Ready Software* (2.^a ed.). Pragmatic Bookshelf.
4. k6.io (2024). *k6 Documentation — Performance Testing for Developers*. <https://k6.io/docs/>
5. FastAPI (2024). *FastAPI Documentation — Testing*. <https://fastapi.tiangolo.com/tutorial/testing/>
6. OWASP Foundation (2023). *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>
7. Humble, J., & Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.
8. Molina Barriga, M. (2026). *Guía de Práctica de Laboratorio 09: Software Quality y Software Testing aplicados a entornos distribuidos*. UNSA, Facultad de Ingeniería de Producción y Servicios.